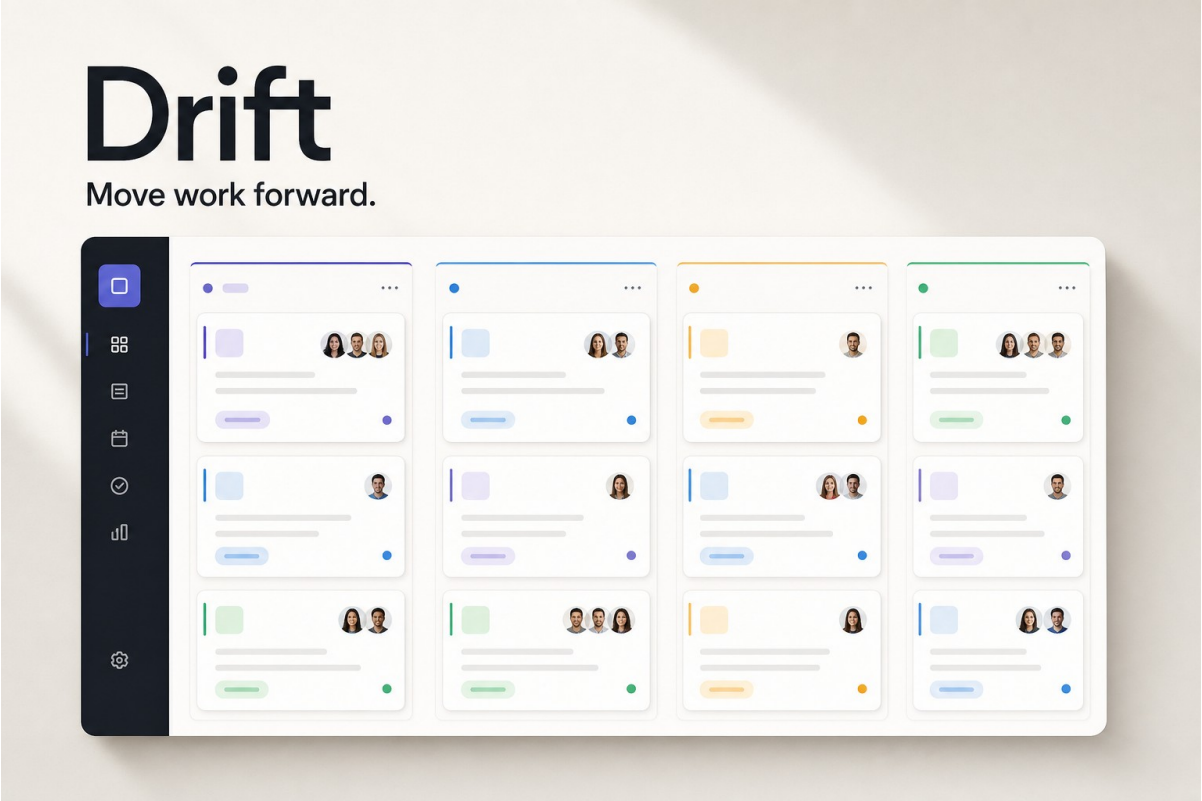
PROJECT DELIVERY REPORT

Drift

A calmer Kanban workspace for focused teams

React + TypeScript + Supabase | Guest auth + RLS | August 2026



Prepared by Pratham Mavle

17 August 2026

Solution at a glance

> OUTCOME
> Drift delivers a polished, team-ready Kanban experience backed by a live Supabase Free Tier project. The public deployment creates an anonymous guest session automatically, persists work in Postgres, enforces per-user access through RLS, and refreshes owned workspace data through Realtime.

Live frontend	Open live app
Public repository	View public repository
Core stack	React 19, TypeScript, Vinext/Vite, Supabase, dnd-kit, Lucide
Delivery status	Public Supabase deployment active; anonymous Auth, RLS persistence, and realtime configured

Overview

Drift is a single-page task workspace designed to feel closer to Linear, Asana, and Notion than a generic todo list. The experience centers the board, keeps secondary controls quiet, and lets users move from scanning to editing without losing context. A deep charcoal rail anchors the interface; warm neutral surfaces, restrained indigo accents, compact cards, and semantic status colors create hierarchy without visual noise.

The frontend calls Supabase directly with its public publishable browser key. The production deployment first restores or creates an anonymous Auth session, then loads only rows allowed by Row Level Security. An explicit browser-local demo path remains available for development, but a failing live connection never silently falls back to local data.

Architecture and data flow

1. Guest identity. Supabase Auth restores the browser session or calls signInAnonymously() once on first launch.
2. Protected data. The React client reads tasks and collaboration records through the public anon key; RLS evaluates auth.uid() for every request.
3. Optimistic interaction. Drag, edit, and create operations update the interface immediately, persist through transactional Supabase RPCs, and roll back visibly on failure.
4. Realtime refresh. Postgres Changes refresh the owned workspace after task, comment, label, assignment, or activity events.

Design decisions

- Board-first hierarchy. The first viewport is the workflow itself, not generic dashboard chrome.
- Calm visual system. Warm ivory canvas, charcoal navigation, indigo focus color, and semantic slate/blue/amber/emerald workflow accents.
- Purposeful density. Metadata stays compact; task titles and status remain the strongest signals.
- Dedicated drag handle. Opening a task and moving it are distinct interactions, reducing accidental drags.
- Progressive detail. The right-side task drawer preserves board context on desktop and becomes a full-screen sheet on mobile.
- Honest state communication. Loading, empty, filtered-empty, reconnecting, mutation error, and demo states are deliberately visible.

Implemented capabilities

Capability	Implementation	Status
Kanban workflow	Four fixed columns, cross-column drag, same-column ordering, keyboard/touch sensors, atomic persistence	Built
Task management	Create, edit, delete, descriptions, priority, due dates, status, ordering	Built
Guest accounts	Automatic Supabase anonymous sign-in with persisted browser session	Built
Team + assignees	User-owned lightweight member profiles and many-to-many task assignments	Advanced
Comments	Chronological task comments with timestamps and realtime refresh	Advanced
Activity log	Append-only trigger-generated history for moves, edits, assignments, labels, and comments	Advanced
Labels + filters	Reusable labels; search plus priority, assignee, and label filters	Advanced
Due-date signals	Overdue, due-soon, later, and completed treatments directly on cards	Advanced
Summary + states	Total/completed/overdue stats; loading, empty, no-results, reconnecting, and error states	Advanced
Responsive UI	Collapsed navigation, horizontal mobile board, touch drag, full-screen detail sheet	Built

Interaction details

- Drag and drop uses pointer activation distance, delayed touch activation, keyboard coordinates, a lifted overlay, destination highlighting, persistent position values, and rollback messaging.

- Search matches task title and description. Priority, assignee, and label filters combine with search, preserve the four-column spatial model, and report the visible result count.
- Due dates are overdue only for incomplete tasks before today; dates within two days are due soon; completed dates are visually subdued.
- Keyboard shortcuts focus search with slash or Command/Ctrl-K and open task creation with N. Global shortcuts pause while a dialog is open.

Responsive behavior

At wide widths all four columns fit beside the workspace rail. The rail collapses on tablet layouts. On mobile, columns become horizontal snap points at roughly 84% of the viewport, task details become full-screen, the search field remains reachable, and the compact workspace menu exposes team management without flattening the board into a generic list.

Supabase security model

> **SECURITY BOUNDARY**
> Frontend visibility is not the authorization layer. Every table is protected in Postgres with RLS, grants, composite owner foreign keys, and ownership validation triggers. The service-role key is never required, stored, or committed.

- Anonymous Supabase users receive the authenticated database role after sign-in; clients without a session keep the anon role and have no table privileges.
- Owned rows store user_id with a default of auth.uid(); policies require the stored value to equal the current session for SELECT, INSERT, UPDATE, and DELETE.
- Join tables store user_id and use composite foreign keys, so tasks cannot be linked to a member or label from another guest workspace.
- SECURITY INVOKER RPCs make task creation, task/relationship edits, and multi-card reorders all-or-nothing while preserving grants and RLS as the authorization boundary.
- Activity history is append-only to browser clients. SECURITY DEFINER triggers generate authoritative entries for creation, updates, moves, assignments, labels, and comments.
- Realtime publication is registered idempotently. RLS SELECT policies remain the subscription boundary.
- Deleting an Auth user cascades through all owned product data; deleting a task cascades through its assignments, labels, comments, and history.

Database entities

Table	Responsibility
tasks	Task content, workflow status, priority, due date, and numeric position.
team_members	User-owned lightweight profiles for visual assignment.
task_assignees	Many-to-many assignments with cross-owner protection.
comments	Chronological task discussion with ownership validation.
labels	Reusable custom names and colors.
task_labels	Many-to-many task tagging with cross-owner protection.
activity_logs	Trigger-authored, append-only task history.

Local setup

Prerequisites: Node.js 22.13 or newer and a free Supabase project.

1. Clone the public repository and install dependencies.
2. Run the complete SQL in Appendix A (also available at supabase/schema.sql) in the Supabase SQL Editor.
3. Enable Anonymous Sign-Ins under Authentication > Providers in the Supabase dashboard.
4. Copy the project URL and public anon/publishable key into .env.local. Never add a service-role key.
5. Start the development server and open http://localhost:3000.

```
git clone https://github.com/pratham-mavle/drift-task-board.git
cd drift-task-board
npm install
cp .env.example .env.local
npm run dev
```

Environment values

```
NEXT_PUBLIC_SUPABASE_URL=https://your-project-ref.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=your-public-anon-key
```

> **DEPLOYMENT NOTE**
> The public Sites deployment is connected to a Supabase Nano Free Tier project in US East (Ohio). Hosting contains only the project URL and public publishable key. Anonymous Auth, RLS persistence, trigger-authored activity, and Realtime are active; no service-role or database credential is deployed.

Verification and delivery

- TypeScript compilation completed with no errors.
- ESLint completed with no errors or warnings.
- Production Vinext/Vite build completed successfully.
- Production-only and full npm security audits both reported zero vulnerabilities.
- Automated worker-rendering and security-contract tests passed (2 of 2).
- The complete migrations were applied successfully to the hosted Supabase project with all seven tables, transactional RPCs, triggers, grants, RLS policies, and Realtime publication entries.
- A live two-session integration test passed anonymous sign-in, owner defaults, User A/User B isolation, cross-owner rollback, atomic creation/edit/reorder, relationship clearing, realtime task updates, activity logs, and cleanup.
- The Supabase-connected public Sites deployment returned HTTP 200 with the expected Drift metadata and application shell.
- Public GitHub repository contains source, setup guidance, environment example, and full schema; no secret environment file or service-role key is committed.

Tradeoffs and next steps

- Lightweight team profiles, not invited identities. This keeps anonymous onboarding simple. A multi-user version would add organizations, invitations, membership roles, and per-board permissions.
- One board per guest. A boards table plus board_id foreign keys would support multiple projects, templates, and archived boards.
- Client-side direct data access is appropriate because RLS remains the security boundary. Narrow SECURITY INVOKER RPCs handle the multi-row workflows; a server API would still help with notifications, external integrations, and rate limiting.
- Reordering atomically rewrites affected integer positions. Fractional ranking would reduce writes on very large columns.
- Plain-text comments. Mentions, rich text, attachments, notification delivery, and threaded replies are natural extensions.
- Automated QA covers compilation, linting, production rendering, schema behavior, and two independent live anonymous sessions. The next layer would drive drag, touch, focus trapping, and realtime reconnection in full browser tests.

Appendix A - Full Supabase database schema

Source: supabase/schema.sql | Run once in the Supabase SQL Editor, then enable Anonymous Sign-Ins.

```
-- Task Board: complete Supabase schema
--
-- Anonymous Supabase users receive the `authenticated` database role after
-- sign-in. Every row is scoped to auth.uid(), and unauthenticated (`anon`)
-- clients receive no table privileges.

begin;

create extension if not exists pgcrypto with schema extensions;

-- ---------------------------------------------------------------------------
-- Core entities
-- ---------------------------------------------------------------------------

create table if not exists public.tasks (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null default auth.uid()
    references auth.users (id) on delete cascade,
  title text not null,
  description text,
  status text not null default 'todo',
  priority text not null default 'normal',
  due_date date,
  -- Fractional positions let the client insert a card between two cards
  -- without rewriting the entire column.
  position numeric(20, 6) not null default 0,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),

  constraint tasks_id_user_id_key unique (id, user_id),
  constraint tasks_title_not_blank
    check (char_length(btrim(title)) between 1 and 240),
  constraint tasks_description_length
    check (description is null or char_length(description) <= 20000),
  constraint tasks_status_valid
    check (status in ('todo', 'in_progress', 'in_review', 'done')),
  constraint tasks_priority_valid
    check (priority in ('low', 'normal', 'high'))
);

create table if not exists public.team_members (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null default auth.uid()
    references auth.users (id) on delete cascade,
  name text not null,
  avatar_url text,
  color text not null default '#64748B',
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),

  constraint team_members_id_user_id_key unique (id, user_id),
  constraint team_members_name_not_blank
    check (char_length(btrim(name)) between 1 and 100),
  constraint team_members_avatar_url_length
    check (avatar_url is null or char_length(avatar_url) <= 2048),
  constraint team_members_color_hex
    check (color ~ '^#[0-9A-Fa-f]{6}$')
```

```
);

create table if not exists public.labels (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null default auth.uid()
    references auth.users (id) on delete cascade,
  name text not null,
  color text not null default '#6366F1',
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),

  constraint labels_id_user_id_key unique (id, user_id),
  constraint labels_name_not_blank
    check (char_length(btrim(name)) between 1 and 40),
  constraint labels_color_hex
    check (color ~ '^#[0-9A-Fa-f]{6}$')
);

-- A guest may use the same label name with different casing only once.
create unique index if not exists labels_user_name_unique
  on public.labels (user_id, lower(name));

-- ---------------------------------------------------------------------------
-- Task relationships and collaboration data
--
-- Each relationship stores user_id and uses composite foreign keys. This is
-- deliberate: even privileged/import code cannot link a task owned by one
-- guest to a member or label owned by another guest.
-- ---------------------------------------------------------------------------

create table if not exists public.task_assignees (
  task_id uuid not null,
  team_member_id uuid not null,
  user_id uuid not null default auth.uid()
    references auth.users (id) on delete cascade,
  created_at timestamptz not null default now(),

  primary key (task_id, team_member_id),
  constraint task_assignees_task_owner_fkey
    foreign key (task_id, user_id)
    references public.tasks (id, user_id) on delete cascade,
  constraint task_assignees_member_owner_fkey
    foreign key (team_member_id, user_id)
    references public.team_members (id, user_id) on delete cascade
);

create table if not exists public.comments (
  id uuid primary key default gen_random_uuid(),
  task_id uuid not null,
  user_id uuid not null default auth.uid()
    references auth.users (id) on delete cascade,
  body text not null,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),

  constraint comments_id_user_id_key unique (id, user_id),
  constraint comments_task_owner_fkey
    foreign key (task_id, user_id)
    references public.tasks (id, user_id) on delete cascade,
  constraint comments_body_not_blank
    check (char_length(btrim(body)) between 1 and 10000)
);
```

```
create table if not exists public.task_labels (
  task_id uuid not null,
  label_id uuid not null,
  user_id uuid not null default auth.uid()
    references auth.users (id) on delete cascade,
  created_at timestamptz not null default now(),

  primary key (task_id, label_id),
  constraint task_labels_task_owner_fkey
    foreign key (task_id, user_id)
    references public.tasks (id, user_id) on delete cascade,
  constraint task_labels_label_owner_fkey
    foreign key (label_id, user_id)
    references public.labels (id, user_id) on delete cascade
);

-- Activity rows are append-only to API clients. Database triggers below write
-- authoritative history for task edits, moves, comments, labels and assignees.
create table if not exists public.activity_logs (
  id uuid primary key default gen_random_uuid(),
  task_id uuid not null,
  user_id uuid not null,
  action text not null,
  old_status text,
  new_status text,
  metadata jsonb not null default '{}'::jsonb,
  created_at timestamptz not null default now(),

  constraint activity_logs_task_owner_fkey
    foreign key (task_id, user_id)
    references public.tasks (id, user_id) on delete cascade,
  constraint activity_logs_action_valid check (
    action in (
      'task_created',
      'task_updated',
      'status_changed',
      'assignee_added',
      'assignee_removed',
      'comment_added',
      'label_added',
      'label_removed'
    )
  ),
  constraint activity_logs_old_status_valid check (
    old_status is null
    or old_status in ('todo', 'in_progress', 'in_review', 'done')
  ),
  constraint activity_logs_new_status_valid check (
    new_status is null
    or new_status in ('todo', 'in_progress', 'in_review', 'done')
  ),
  constraint activity_logs_metadata_object
    check (jsonb_typeof(metadata) = 'object')
);

-- ------------------------------------------------------------------------
-- Indexes for board loading, filtering and detail views
-- ------------------------------------------------------------------------

create index if not exists tasks_user_status_position_idx
  on public.tasks (user_id, status, position, created_at);

create index if not exists tasks_user_priority_idx
```

```
  on public.tasks (user_id, priority);

create index if not exists tasks_user_due_date_idx
  on public.tasks (user_id, due_date)
  where due_date is not null and status <> 'done';

create index if not exists tasks_user_created_at_idx
  on public.tasks (user_id, created_at desc);

create index if not exists team_members_user_created_at_idx
  on public.team_members (user_id, created_at);

create index if not exists task_assignees_user_member_idx
  on public.task_assignees (user_id, team_member_id, task_id);

create index if not exists comments_user_task_created_at_idx
  on public.comments (user_id, task_id, created_at);

create index if not exists labels_user_created_at_idx
  on public.labels (user_id, created_at);

create index if not exists task_labels_user_label_idx
  on public.task_labels (user_id, label_id, task_id);

create index if not exists activity_logs_user_task_created_at_idx
  on public.activity_logs (user_id, task_id, created_at desc);

-- ------------------------------------------------------------------------
-- Timestamp and relationship-integrity triggers
-- ------------------------------------------------------------------------

create or replace function public.set_updated_at()
returns trigger
language plpgsql
set search_path = pg_catalog
as $$
begin
  new.updated_at := now();
  return new;
end;
$$;

drop trigger if exists tasks_set_updated_at on public.tasks;
create trigger tasks_set_updated_at
before update on public.tasks
for each row execute function public.set_updated_at();

drop trigger if exists team_members_set_updated_at on public.team_members;
create trigger team_members_set_updated_at
before update on public.team_members
for each row execute function public.set_updated_at();

drop trigger if exists labels_set_updated_at on public.labels;
create trigger labels_set_updated_at
before update on public.labels
for each row execute function public.set_updated_at();

drop trigger if exists comments_set_updated_at on public.comments;
create trigger comments_set_updated_at
before update on public.comments
for each row execute function public.set_updated_at();

-- These friendly validation triggers duplicate the composite FKs intentionally:
```

```
-- they reject cross-owner links before constraint evaluation and return a clear
-- error to direct API clients.
create or replace function public.validate_task_assignee_ownership()
returns trigger
language plpgsql
set search_path = pg_catalog
as $$
begin
  if not exists (
    select 1
    from public.tasks as task
    where task.id = new.task_id and task.user_id = new.user_id
  ) then
    raise exception using
      errcode = '23514',
      message = 'Task does not belong to the assignment owner';
  end if;

  if not exists (
    select 1
    from public.team_members as member
    where member.id = new.team_member_id and member.user_id = new.user_id
  ) then
    raise exception using
      errcode = '23514',
      message = 'Team member does not belong to the assignment owner';
  end if;

  return new;
end;
$$;

drop trigger if exists task_assignees_validate_ownership
  on public.task_assignees;
create trigger task_assignees_validate_ownership
before insert or update on public.task_assignees
for each row execute function public.validate_task_assignee_ownership();

create or replace function public.validate_task_label_ownership()
returns trigger
language plpgsql
set search_path = pg_catalog
as $$
begin
  if not exists (
    select 1
    from public.tasks as task
    where task.id = new.task_id and task.user_id = new.user_id
  ) then
    raise exception using
      errcode = '23514',
      message = 'Task does not belong to the label relationship owner';
  end if;

  if not exists (
    select 1
    from public.labels as label
    where label.id = new.label_id and label.user_id = new.user_id
  ) then
    raise exception using
      errcode = '23514',
      message = 'Label does not belong to the relationship owner';
  end if;
```

```
  return new;
end;
$$;

drop trigger if exists task_labels_validate_ownership on public.task_labels;
create trigger task_labels_validate_ownership
before insert or update on public.task_labels
for each row execute function public.validate_task_label_ownership();

create or replace function public.validate_comment_ownership()
returns trigger
language plpgsql
set search_path = pg_catalog
as $$
begin
  if not exists (
    select 1
    from public.tasks as task
    where task.id = new.task_id and task.user_id = new.user_id
  ) then
    raise exception using
      errcode = '23514',
      message = 'Task does not belong to the comment owner';
  end if;

  return new;
end;
$$;

drop trigger if exists comments_validate_ownership on public.comments;
create trigger comments_validate_ownership
before insert or update on public.comments
for each row execute function public.validate_comment_ownership();

-- ------------------------------------------------------------------------
-- Automatic, append-only activity history
-- ------------------------------------------------------------------------

create or replace function public.log_task_activity()
returns trigger
language plpgsql
security definer
set search_path = pg_catalog
as $$
declare
  changed_fields text[] := array[]::text[];
begin
  if tg_op = 'INSERT' then
    insert into public.activity_logs (
      task_id,
      user_id,
      action,
      new_status,
      metadata
    ) values (
      new.id,
      new.user_id,
      'task_created',
      new.status,
      jsonb_build_object('title', new.title)
    );
    return new;
```

```
  end if;

  if old.status is distinct from new.status then
    insert into public.activity_logs (
      task_id,
      user_id,
      action,
      old_status,
      new_status,
      metadata
    ) values (
      new.id,
      new.user_id,
      'status_changed',
      old.status,
      new.status,
      '{}'::jsonb
    );
  end if;

  if old.title is distinct from new.title then
    changed_fields := array_append(changed_fields, 'title');
  end if;
  if old.description is distinct from new.description then
    changed_fields := array_append(changed_fields, 'description');
  end if;
  if old.priority is distinct from new.priority then
    changed_fields := array_append(changed_fields, 'priority');
  end if;
  if old.due_date is distinct from new.due_date then
    changed_fields := array_append(changed_fields, 'due_date');
  end if;

  if cardinality(changed_fields) > 0 then
    insert into public.activity_logs (
      task_id,
      user_id,
      action,
      metadata
    ) values (
      new.id,
      new.user_id,
      'task_updated',
      jsonb_build_object('fields', to_jsonb(changed_fields))
    );
  end if;

  return new;
end;
$$;

drop trigger if exists tasks_log_activity on public.tasks;
create trigger tasks_log_activity
after insert or update on public.tasks
for each row execute function public.log_task_activity();

create or replace function public.log_assignee_activity()
returns trigger
language plpgsql
security definer
set search_path = pg_catalog
as $$
declare
```

```
  source_task_id uuid;
  source_user_id uuid;
  source_member_id uuid;
  source_action text;
begin
  -- Skip relationship removals performed by an FK cascade (for example when
  -- the task itself is deleted). Direct unassign operations run at depth 1.
  if tg_op = 'DELETE' and pg_trigger_depth() > 1 then
    return old;
  end if;

  if tg_op = 'INSERT' then
    source_task_id := new.task_id;
    source_user_id := new.user_id;
    source_member_id := new.team_member_id;
    source_action := 'assignee_added';
  else
    source_task_id := old.task_id;
    source_user_id := old.user_id;
    source_member_id := old.team_member_id;
    source_action := 'assignee_removed';
  end if;

  -- During a task/user cascade the parent may already be gone; no activity
  -- should be created for a task whose history is being deleted.
  if exists (
    select 1 from public.tasks
    where id = source_task_id and user_id = source_user_id
  ) then
    insert into public.activity_logs (
      task_id,
      user_id,
      action,
      metadata
    ) values (
      source_task_id,
      source_user_id,
      source_action,
      jsonb_build_object('team_member_id', source_member_id)
    );
  end if;

  if tg_op = 'INSERT' then
    return new;
  end if;
  return old;
end;
$$;

drop trigger if exists task_assignees_log_activity
  on public.task_assignees;
create trigger task_assignees_log_activity
after insert or delete on public.task_assignees
for each row execute function public.log_assignee_activity();

create or replace function public.log_label_activity()
returns trigger
language plpgsql
security definer
set search_path = pg_catalog
as $$
declare
  source_task_id uuid;
```

```
  source_user_id uuid;
  source_label_id uuid;
  source_action text;
begin
  if tg_op = 'DELETE' and pg_trigger_depth() > 1 then
    return old;
  end if;

  if tg_op = 'INSERT' then
    source_task_id := new.task_id;
    source_user_id := new.user_id;
    source_label_id := new.label_id;
    source_action := 'label_added';
  else
    source_task_id := old.task_id;
    source_user_id := old.user_id;
    source_label_id := old.label_id;
    source_action := 'label_removed';
  end if;

  if exists (
    select 1 from public.tasks
    where id = source_task_id and user_id = source_user_id
  ) then
    insert into public.activity_logs (
      task_id,
      user_id,
      action,
      metadata
    ) values (
      source_task_id,
      source_user_id,
      source_action,
      jsonb_build_object('label_id', source_label_id)
    );
  end if;

  if tg_op = 'INSERT' then
    return new;
  end if;
  return old;
end;
$$;

drop trigger if exists task_labels_log_activity on public.task_labels;
create trigger task_labels_log_activity
after insert or delete on public.task_labels
for each row execute function public.log_label_activity();

create or replace function public.log_comment_activity()
returns trigger
language plpgsql
security definer
set search_path = pg_catalog
as $$
begin
  insert into public.activity_logs (
    task_id,
    user_id,
    action,
    metadata
  ) values (
    new.task_id,
```

```
    new.user_id,
    'comment_added',
    jsonb_build_object('comment_id', new.id)
  );
  return new;
end;
$$;

drop trigger if exists comments_log_activity on public.comments;
create trigger comments_log_activity
after insert on public.comments
for each row execute function public.log_comment_activity();

-- Trigger functions should not be callable as public RPCs. Triggers continue
-- to execute with these privileges revoked.
revoke all on function public.set_updated_at() from public, anon, authenticated;
revoke all on function public.validate_task_assignee_ownership()
  from public, anon, authenticated;
revoke all on function public.validate_task_label_ownership()
  from public, anon, authenticated;
revoke all on function public.validate_comment_ownership()
  from public, anon, authenticated;
revoke all on function public.log_task_activity()
  from public, anon, authenticated;
revoke all on function public.log_assignee_activity()
  from public, anon, authenticated;
revoke all on function public.log_label_activity()
  from public, anon, authenticated;
revoke all on function public.log_comment_activity()
  from public, anon, authenticated;

-- ------------------------------------------------------------------------
-- Row Level Security
-- ------------------------------------------------------------------------

alter table public.tasks enable row level security;
alter table public.team_members enable row level security;
alter table public.task_assignees enable row level security;
alter table public.comments enable row level security;
alter table public.activity_logs enable row level security;
alter table public.labels enable row level security;
alter table public.task_labels enable row level security;

drop policy if exists tasks_owner_access on public.tasks;
create policy tasks_owner_access
on public.tasks
for all
to authenticated
using (user_id = (select auth.uid()))
with check (user_id = (select auth.uid()));

drop policy if exists team_members_owner_access on public.team_members;
create policy team_members_owner_access
on public.team_members
for all
to authenticated
using (user_id = (select auth.uid()))
with check (user_id = (select auth.uid()));

drop policy if exists labels_owner_access on public.labels;
create policy labels_owner_access
on public.labels
for all
```

```
to authenticated
using (user_id = (select auth.uid()))
with check (user_id = (select auth.uid()));

drop policy if exists task_assignees_owner_access
  on public.task_assignees;
create policy task_assignees_owner_access
on public.task_assignees
for all
to authenticated
using (
  user_id = (select auth.uid())
  and exists (
    select 1 from public.tasks as task
    where task.id = task_assignees.task_id
      and task.user_id = (select auth.uid())
  )
  and exists (
    select 1 from public.team_members as member
    where member.id = task_assignees.team_member_id
      and member.user_id = (select auth.uid())
  )
)
with check (
  user_id = (select auth.uid())
  and exists (
    select 1 from public.tasks as task
    where task.id = task_assignees.task_id
      and task.user_id = (select auth.uid())
  )
  and exists (
    select 1 from public.team_members as member
    where member.id = task_assignees.team_member_id
      and member.user_id = (select auth.uid())
  )
);

drop policy if exists comments_owner_access on public.comments;
create policy comments_owner_access
on public.comments
for all
to authenticated
using (
  user_id = (select auth.uid())
  and exists (
    select 1 from public.tasks as task
    where task.id = comments.task_id
      and task.user_id = (select auth.uid())
  )
)
with check (
  user_id = (select auth.uid())
  and exists (
    select 1 from public.tasks as task
    where task.id = comments.task_id
      and task.user_id = (select auth.uid())
  )
);

drop policy if exists task_labels_owner_access on public.task_labels;
create policy task_labels_owner_access
on public.task_labels
for all
```

```
to authenticated
using (
  user_id = (select auth.uid())
  and exists (
    select 1 from public.tasks as task
    where task.id = task_labels.task_id
      and task.user_id = (select auth.uid())
  )
  and exists (
    select 1 from public.labels as label
    where label.id = task_labels.label_id
      and label.user_id = (select auth.uid())
  )
)
with check (
  user_id = (select auth.uid())
  and exists (
    select 1 from public.tasks as task
    where task.id = task_labels.task_id
      and task.user_id = (select auth.uid())
  )
  and exists (
    select 1 from public.labels as label
    where label.id = task_labels.label_id
      and label.user_id = (select auth.uid())
  )
);

-- Audit history is readable but not directly writable by browser clients.
drop policy if exists activity_logs_owner_read on public.activity_logs;
create policy activity_logs_owner_read
on public.activity_logs
for select
to authenticated
using (
  user_id = (select auth.uid())
  and exists (
    select 1 from public.tasks as task
    where task.id = activity_logs.task_id
      and task.user_id = (select auth.uid())
  )
);

-- Explicit grants complement RLS. A Supabase anonymous-auth session uses the
-- authenticated role; a client without a session uses anon and has no access.
revoke all on table public.tasks from public, anon;
revoke all on table public.team_members from public, anon;
revoke all on table public.task_assignees from public, anon;
revoke all on table public.comments from public, anon;
revoke all on table public.activity_logs from public, anon;
revoke all on table public.labels from public, anon;
revoke all on table public.task_labels from public, anon;

grant usage on schema public to authenticated;
grant select, insert, update, delete on table public.tasks to authenticated;
grant select, insert, update, delete on table public.team_members to authenticated;
grant select, insert, update, delete on table public.task_assignees to authenticated;
grant select, insert, update, delete on table public.comments to authenticated;
grant select on table public.activity_logs to authenticated;
grant select, insert, update, delete on table public.labels to authenticated;
grant select, insert, update, delete on table public.task_labels to authenticated;

-- ------------------------------------------------------------------------
```

```
-- Atomic task mutation RPCs
--
-- These functions run with the caller's privileges. The authenticated role's
-- table grants and RLS policies therefore remain the authorization boundary.
-- Any exception aborts the entire RPC transaction, so callers never observe a
-- partially reordered board, partially synchronized task, or incomplete task.
-- ----------------------------------------------------------------------------

create or replace function public.reorder_tasks(p_updates jsonb)
returns setof public.tasks
language plpgsql
volatile
security invoker
set search_path = pg_catalog
as $$
declare
  current_user_id uuid := auth.uid();
  requested_count integer;
  affected_count integer;
begin
  if current_user_id is null then
    raise exception using
      errcode = '42501',
      message = 'An authenticated user is required';
  end if;

  if p_updates is null or jsonb_typeof(p_updates) <> 'array' then
    raise exception using
      errcode = '22023',
      message = 'p_updates must be a JSON array';
  end if;

  select count(*)
  into requested_count
  from jsonb_array_elements(p_updates);

  if exists (
    select 1
    from jsonb_array_elements(p_updates) as entry(value)
    where jsonb_typeof(value) <> 'object'
      or not (value ? 'id')
      or jsonb_typeof(value -> 'id') <> 'string'
      or not (value ? 'status')
      or jsonb_typeof(value -> 'status') <> 'string'
      or value ->> 'status' not in (
        'todo',
        'in_progress',
        'in_review',
        'done'
      )
      or not (value ? 'position')
      or jsonb_typeof(value -> 'position') <> 'number'
  ) then
    raise exception using
      errcode = '22023',
      message = 'Each update must contain string id, valid status, and numeric position';
  end if;

  if requested_count <> (
    select count(distinct value ->> 'id')
    from jsonb_array_elements(p_updates) as entry(value)
  ) then
    raise exception using
```

```
      errcode = '22023',
      message = 'p_updates must not contain duplicate task ids';
  end if;

  return query
  with requested as (
    select update_row.id, update_row.status, update_row.position
    from jsonb_to_recordset(p_updates) as update_row(
      id uuid,
      status text,
      position numeric
    )
  ),
  changed as (
    update public.tasks as task
    set
      status = requested.status,
      position = requested.position
    from requested
    where task.id = requested.id
      and task.user_id = current_user_id
    returning task.*
  )
  select changed.*
  from changed;

  get diagnostics affected_count = row_count;

  if affected_count <> requested_count then
    raise exception using
      errcode = 'P0002',
      message = 'One or more tasks were not found or are not owned by the current user';
  end if;
end;
$$;

create or replace function public.create_task_with_relationships(
  p_title text,
  p_description text,
  p_status text,
  p_priority text,
  p_due_date date,
  p_position numeric,
  p_assignee_ids uuid[] default array[]::uuid[],
  p_label_ids uuid[] default array[]::uuid[]
)
returns setof public.tasks
language plpgsql
volatile
security invoker
set search_path = pg_catalog
as $$
declare
  current_user_id uuid := auth.uid();
  requested_assignee_ids uuid[] := coalesce(
    p_assignee_ids,
    array[]::uuid[]
  );
  requested_label_ids uuid[] := coalesce(
    p_label_ids,
    array[]::uuid[]
  );
  created_task public.tasks%rowtype;
```

```
begin
  if current_user_id is null then
    raise exception using
      errcode = '42501',
      message = 'An authenticated user is required';
  end if;

  if exists (
    select 1
    from unnest(requested_assignee_ids) as requested(member_id)
    left join public.team_members as member
      on member.id = requested.member_id
      and member.user_id = current_user_id
    where member.id is null
  ) then
    raise exception using
      errcode = '23514',
      message = 'Every assignee must belong to the current user';
  end if;

  if exists (
    select 1
    from unnest(requested_label_ids) as requested(label_id)
    left join public.labels as label
      on label.id = requested.label_id
      and label.user_id = current_user_id
    where label.id is null
  ) then
    raise exception using
      errcode = '23514',
      message = 'Every label must belong to the current user';
  end if;

  insert into public.tasks (
    user_id,
    title,
    description,
    status,
    priority,
    due_date,
    position
  ) values (
    current_user_id,
    p_title,
    p_description,
    p_status,
    p_priority,
    p_due_date,
    p_position
  )
  returning * into created_task;

  insert into public.task_assignees (
    task_id,
    team_member_id,
    user_id
  )
  select
    created_task.id,
    requested.member_id,
    current_user_id
  from (
    select distinct unnest(requested_assignee_ids) as member_id
```

```
  ) as requested;

  insert into public.task_labels (
    task_id,
    label_id,
    user_id
  )
  select
    created_task.id,
    requested.label_id,
    current_user_id
  from (
    select distinct unnest(requested_label_ids) as label_id
  ) as requested;

  return query
  select task.*
  from public.tasks as task
  where task.id = created_task.id
    and task.user_id = current_user_id;
end;
$$;


create or replace function public.update_task_with_relationships(
  p_task_id uuid,
  p_title text,
  p_description text,
  p_status text,
  p_priority text,
  p_due_date date,
  p_position numeric,
  p_assignee_ids uuid[] default array[]::uuid[],
  p_label_ids uuid[] default array[]::uuid[]
)
returns setof public.tasks
language plpgsql
volatile
security invoker
set search_path = pg_catalog
as $$
declare
  current_user_id uuid := auth.uid();
  requested_assignee_ids uuid[] := coalesce(
    p_assignee_ids,
    array[]::uuid[]
  );
  requested_label_ids uuid[] := coalesce(
    p_label_ids,
    array[]::uuid[]
  );
  updated_task public.tasks%rowtype;
begin
  if current_user_id is null then
    raise exception using
      errcode = '42501',
      message = 'An authenticated user is required';
  end if;

  update public.tasks as task
  set
    title = p_title,
    description = p_description,
```

```
  status = p_status,
  priority = p_priority,
  due_date = p_due_date,
  position = p_position
where task.id = p_task_id
  and task.user_id = current_user_id
returning task.* into updated_task;

if not found then
  raise exception using
    errcode = 'P0002',
    message = 'Task was not found or is not owned by the current user';
end if;

if exists (
  select 1
  from unnest(requested_assignee_ids) as requested(member_id)
  left join public.team_members as member
    on member.id = requested.member_id
    and member.user_id = current_user_id
  where member.id is null
) then
  raise exception using
    errcode = '23514',
    message = 'Every assignee must belong to the current user';
end if;

if exists (
  select 1
  from unnest(requested_label_ids) as requested(label_id)
  left join public.labels as label
    on label.id = requested.label_id
    and label.user_id = current_user_id
  where label.id is null
) then
  raise exception using
    errcode = '23514',
    message = 'Every label must belong to the current user';
end if;

delete from public.task_assignees as assignment
where assignment.task_id = p_task_id
  and assignment.user_id = current_user_id
  and not (
    assignment.team_member_id = any(requested_assignee_ids)
  );

insert into public.task_assignees (
  task_id,
  team_member_id,
  user_id
)
select
  p_task_id,
  requested.member_id,
  current_user_id
from (
  select distinct unnest(requested_assignee_ids) as member_id
) as requested
on conflict (task_id, team_member_id) do nothing;

delete from public.task_labels as task_label
where task_label.task_id = p_task_id
```

```
    and task_label.user_id = current_user_id
    and not (task_label.label_id = any(requested_label_ids));

  insert into public.task_labels (
    task_id,
    label_id,
    user_id
  )
  select
    p_task_id,
    requested.label_id,
    current_user_id
  from (
    select distinct unnest(requested_label_ids) as label_id
  ) as requested
  on conflict (task_id, label_id) do nothing;

  return query
  select task.*
  from public.tasks as task
  where task.id = p_task_id
    and task.user_id = current_user_id;
end;
$$;

comment on function public.reorder_tasks(jsonb) is
  'Atomically updates status and position for an owned batch of tasks.';

comment on function public.create_task_with_relationships(
  text,
  text,
  text,
  text,
  date,
  numeric,
  uuid[],
  uuid[]
) is
  'Atomically creates an owned task with its assignees and labels.';

comment on function public.update_task_with_relationships(
  uuid,
  text,
  text,
  text,
  text,
  date,
  numeric,
  uuid[],
  uuid[]
) is
  'Atomically edits an owned task and synchronizes its assignees and labels.';

revoke all on function public.reorder_tasks(jsonb)
  from public, anon, authenticated, service_role;
revoke all on function public.create_task_with_relationships(
  text,
  text,
  text,
  text,
  date,
  numeric,
  uuid[],
```

```
  uuid[]
) from public, anon, authenticated, service_role;
revoke all on function public.update_task_with_relationships(
  uuid,
  text,
  text,
  text,
  text,
  date,
  numeric,
  uuid[],
  uuid[]
) from public, anon, authenticated, service_role;

grant execute on function public.reorder_tasks(jsonb) to authenticated;
grant execute on function public.create_task_with_relationships(
  text,
  text,
  text,
  text,
  date,
  numeric,
  uuid[],
  uuid[]
) to authenticated;
grant execute on function public.update_task_with_relationships(
  uuid,
  text,
  text,
  text,
  text,
  date,
  numeric,
  uuid[],
  uuid[]
) to authenticated;

-- ------------------------------------------------------------------------
-- Realtime
--
-- Supabase normally creates supabase_realtime. The guarded block also supports
-- a fresh local Postgres instance and does not fail if a table was added before.
-- RLS SELECT policies remain the authorization boundary for subscriptions.
-- ------------------------------------------------------------------------

alter table public.tasks replica identity full;
alter table public.team_members replica identity full;
alter table public.task_assignees replica identity full;
alter table public.comments replica identity full;
alter table public.activity_logs replica identity full;
alter table public.labels replica identity full;
alter table public.task_labels replica identity full;

do $$
declare
  relation_name text;
  realtime_relations constant text[] := array[
    'tasks',
    'team_members',
    'task_assignees',
    'comments',
    'activity_logs',
    'labels',
```

```
    'task_labels'
  ];
begin
  if not exists (
    select 1 from pg_catalog.pg_publication
    where pubname = 'supabase_realtime'
  ) then
    execute 'create publication supabase_realtime';
  end if;

  foreach relation_name in array realtime_relations loop
    if not exists (
      select 1
      from pg_catalog.pg_publication_tables
      where pubname = 'supabase_realtime'
        and schemaname = 'public'
        and tablename = relation_name
    ) then
      execute format(
        'alter publication supabase_realtime add table public.%I',
        relation_name
      );
    end if;
  end loop;
end;
$$;

commit;
```